Q) Assignment : Vector and Matrix Operations Design parallel algorithm to Add two large vectors. Multiply Vector and Matrix. Multiply two N × N arrays using n2 processors.

```
%%cu

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cuda.h>
#define N 4
#define TPB 2

// CUDA kernel. Each thread takes care of one element of c
__global__ void vecAdd(double *a, double *b, double *c, int n){
    // Get our global thread ID
    int id = blockIdx.x*blockDim.x+threadIdx.x;
    // Make sure we do not go out of bounds
    if (id < n)
        c[id] = a[id] + b[id];
}

__global__ void matrixMultiplication(int *a, int *b, int *c, int n){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int sum = 0;
    int i;
    if (row < n && col < n){
      for( i=0; i<n ;i++){
        sum += a[row *n +i] *  b[i * n+ col];
      }
      c[row *N +col]=sum;
    }
}

void vecAdd(){
    // Size of vectors
    int n = 100000;

    // Host input vectors
    double *h_a;
    double *h_b;
    //Host output vector
    double *h_c;

    // Device input vectors
    double *d_a;
    double *d_b;
    //Device output vector
    double *d_c;

    // Size, in bytes, of each vector
    size_t bytes = n*sizeof(double);

    // Allocate memory for each vector on host
    h_a = (double*)malloc(bytes);
```

```
    h_b = (double*)malloc(bytes);
    h_c = (double*)malloc(bytes);

    // Allocate memory for each vector on GPU
    cudaMalloc(&d_a, bytes);
    cudaMalloc(&d_b, bytes);
    cudaMalloc(&d_c, bytes);

    int i;
    // Initialize vectors on host
    for( i = 0; i < n; i++ ){
        h_a[i] = i;
        h_b[i] = i;
    }

    // Copy host vectors to device
    cudaMemcpy( d_a, h_a, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy( d_b, h_b, bytes, cudaMemcpyHostToDevice);

    int blockSize, gridSize;

    // Number of threads in each thread block
    blockSize = 1024;

    // Number of thread blocks in grid
    gridSize = (int)ceil((float)n/blockSize);

    // Execute the kernel
    vecAdd<<<gridSize, blockSize>>>(d_a, d_b, d_c, n);

    // Copy array back to host
    cudaMemcpy( h_c, d_c, bytes, cudaMemcpyDeviceToHost );

    // Sum up vector c and print result divided by n, this should equal 1 within error
    double sum = 0;
    for(i=0; i<n; i++)
        sum += h_c[i];
    printf("Sum: %f\n", sum/n);

    // Release device memory
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);

    // Release host memory
    free(h_a);
    free(h_b);
    free(h_c);
}

void matMul(){
  int *h_a, *h_b, *h_c;
  int *d_a, *d_b, *d_c;

  int size=sizeof(int)*N*N;
  cudaEvent_t start,end;
  float time=0;
  h_a=(int*)malloc(size);
  h_b=(int*)malloc(size);
  h_c=(int*)malloc(size);
```

```
cudaEventCreate(&start);
cudaEventCreate(&end);
cudaEventRecord(start);

cudaMalloc(&d_a, size);
cudaMalloc(&d_b, size);
cudaMalloc(&d_c, size);

for (int i = 0; i < N*N; i++){
        h_a[i] = random() % N;
        h_b[i] = random() % N;
}

printf("\nMatrix A =>\n\n");
for (int i = 0; i < N; i++){
  for(int j = 0;j<N; j++){
    printf("%d ",h_a[i*N + j]);
  }
        printf("\n");
 }

printf("\nMatrix B =>\n\n");
 for (int i = 0; i < N; i++){
    for(int j = 0;j<N; j++){
      printf("%d ",h_b[i*N + j]);
    }
    printf("\n");
  }



cudaMemcpy( d_a, h_a, size, cudaMemcpyHostToDevice);
cudaMemcpy( d_b, h_b, size, cudaMemcpyHostToDevice);

int BLOCK_SIZE=N / TPB;

dim3 GridSize(BLOCK_SIZE, BLOCK_SIZE);
dim3 BlockSize(TPB, TPB);

matrixMultiplication<<<GridSize,BlockSize>>>(d_a, d_b, d_c, N);

cudaMemcpy( h_c, d_c, size, cudaMemcpyDeviceToHost );
cudaEventRecord(end);
cudaEventSynchronize(end);
cudaEventElapsedTime(&time,start,end);
printf("\nMatrix C =>\n\n");

for (int i = 0; i < N; i++){
  for(int j = 0;j<N; j++){
    printf("%d ",h_c[i*N + j]);
  }
  printf("\n");
}

printf("Time taken to perform %d by %d matrix mul is: %lf ms",N,N,time);

 cudaFree(d_a);
 cudaFree(d_b);
 cudaFree(d_c);
```

```
   free(h_a);
   free(h_b);
   free(h_c);
}


int main( int argc, char* argv[] )
{
    printf("Add two large vectors \n");
    vecAdd();
    //printf("\n\n\n Multiply Vector and Matrix \n");
    printf("\n\n\n Multiply two N × N arrays using n2 processors\n");
    matMul();
 return 0;
}
```